PROGRESS REPORT

Ghana Smart Hospital Service Operations Optimizer

Course: DCIT 204/308 – Joint Data Structures and Algorithms Semester Project
Team: Charismatic Enigmas Inner Group 1
Date: 25 August 2026
Project Type: Console-Based Data Structures and Algorithms
Application **Context:** Hospital and clinic service operations in Greater Accra, Ghana

1. Introduction

The Ghana Smart Hospital Service Operations Optimizer is a console-based application designed to model and analyze service operations across a network of healthcare facilities in Greater Accra. The system focuses on operational activities such as ambulance dispatch, patient referrals, pharmacy runs, laboratory sample transportation, equipment movement, and porter assignments.

The project applies fundamental data structures and algorithms to a realistic Ghanaian healthcare context. The system uses a dataset representing ten healthcare facilities and their surrounding road network. SQLite is used for persistent storage, while the core data structures and algorithms are implemented independently rather than relying on Java's built-in collection classes for assessed functionality.

The application is being developed using Java 17, Maven, SQLite, JDBC, and JUnit 5.

2. Project Objectives

The primary objectives of the project are to:

1. Model healthcare service operations using a realistic local dataset.
2. Implement and demonstrate custom data structures required by the project brief.
3. Apply searching, sorting, graph, greedy, and dynamic programming algorithms to practical operational problems.
4. Store and retrieve application data using SQLite.
5. Evaluate the correctness and efficiency of the implemented algorithms.
6. Provide evidence of correctness through testing, trace tables, proof sketches, and counterexamples.
7. Demonstrate the complete system through an interactive console application.

3. Progress Achieved

3.1 Local Dataset and Ghanaian Context

The local dataset and operational context have been completed.

The system currently models ten healthcare facilities in Greater Accra:

- University of Ghana Hospital, Legon
- Korle-Bu Teaching Hospital

- Ridge Hospital
- 37 Military Hospital
- La General Hospital
- Tema General Hospital
- Madina Polyclinic
- Achimota Hospital
- Nyaho Medical Centre
- Dodowa Health Centre

The dataset contains more records than the minimum requirements specified in the project brief:

Dataset	Records	Minimum Required
Locations	100	50
Roads	370	100
Service Requests	300	Not specified
Resources	30	Not specified
Algorithm Runs	30 seed + live records	Not specified
Total	**830+**	—

Facility coordinates were verified using public geospatial sources. The road network was subsequently revised so that road distances are derived from the actual geographic coordinates rather than being assigned arbitrary random weights.

The dataset generation process is deterministic, allowing the experiments and demonstrations to be reproduced consistently.

3.2 Database Integration

The SQLite database layer has been completed and tested.

The database contains the following six core tables:

1. locations
2. roads
3. service_requests
4. resources
5. algorithm_runs
6. audit_events

The database schema has been implemented in db/schema.sql, while database access is handled through the application's JDBC layer.
The seeded CSV data has also been successfully imported into the SQLite database, and database loading and persistence operations have been verified.

Supporting documentation has been prepared in the form of a data dictionary and an evidence note describing the source and preparation of the dataset.

3.3 Custom Data Structures

The required custom data structure library has been completed.

The implementation currently contains:

1. Dynamic Array List
2. Doubly Linked List
3. Stack
4. Circular Queue
5. Custom Deque
6. Binary Search Tree
7. Red-Black Tree using Left-Leaning Red-Black Tree rotations
8. B-Tree
9. Min-Heap Priority Queue
10. Hash Table
11. Custom Hash Set
12. Disjoint Set
13. Graph adjacency structure

The structures were implemented independently for the assessed core logic rather than relying on Java collection classes.

Each major structure has been tested using normal cases, boundary conditions, and invalid-input cases.

3.4 Algorithms

The main algorithmic components of the project have been implemented and integrated with the application.

Searching

- Linear Search
- Binary Search

Sorting

- Merge Sort
- Quick Sort

- Insertion Sort *Graph Algorithms*
- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Dijkstra's Shortest Path
- Prim's Minimum Spanning Tree
- Kruskal's Minimum Spanning Tree
- Graph cycle detection

Resource Assignment

A greedy resource-assignment algorithm has been implemented to allocate available resources to service requests.

A valid counterexample has also been documented to demonstrate a situation in which the greedy approach does not produce the optimal result.

Dynamic Programming

A dynamic programming solution has been implemented for selecting service requests under a specified resource or budget constraint.

3.5 Console Application

The implemented structures and algorithms have been integrated into a single console application.

The application currently provides a 20-option menu through which the user can interact with the system and execute demonstrations against the loaded dataset.

The menu provides access to the major data structures, algorithms, searches, sorting operations, graph operations, resource assignment, and dynamic programming functionality.

Member-index-derived parameters have also been implemented to ensure that required demonstrations can use parameters derived from team-member information where required by the project specification.

4. Testing and Correctness Verification

Testing and correctness verification have been completed to a substantial level.

The current automated test suite contains **77 tests, all of which are passing**.

Testing Status

Area	Status
Data structure tests	Complete

Algorithm tests	Complete
Boundary-condition tests	Complete
Invalid-input tests	Complete
Database integration	Verified
Acceptance tests	Complete
Total automated tests	77
Passing tests	**77/77**

Additional correctness evidence has been produced, including:

- Trace tables for six required operations
- Three proof sketches
- Two documented counterexamples
- Pseudocode for six major operations
- Red-Black Tree rotation walkthrough
- B-Tree operation traces

The trace tables are generated programmatically using the current dataset to reduce the possibility of documentation becoming inconsistent with the implementation.

5. Efficiency Analysis

The efficiency analysis required for the project has also been completed.

An experiment runner was developed to evaluate:

- Searching performance
- Sorting performance
- Hash table load-factor behaviour
- Binary Search Tree versus balanced-tree performance
- Heap-based dispatch
- Graph algorithm performance

Each experiment uses three repeated runs for each input size.

The experimental results have been exported to:

efficiency_results.csv

Six required graphs have also been generated and stored in docs/graphs/.

The graphs were generated as dependency-free SVG files to improve portability and reduce dependency on additional visualization libraries.

6. Problems Identified and Resolved

Several issues were identified during development and corrected.

6.1 Unrealistic Road Weights

The initial road-generation process assigned random distances that were not related to the geographic positions of the facilities.
This caused unrealistic shortest-path results. For example, a short intra-campus journey could incorrectly be assigned a route that travelled across a large portion of the city.

The road-generation process was therefore redesigned to derive distances from facility coordinates using the Haversine distance calculation. Connectivity was also generated using minimum-spanning-tree-based logic.

The revised implementation was subsequently tested, and all 77 automated tests continued to pass.

6.2 Coordinate Accuracy

The initial facility coordinates contained approximations that were unsuitable for realistic routing.

The coordinates were replaced with verified values obtained from public geospatial sources. An additional issue involving the coordinate jitter radius was identified and corrected, reducing the generated variation from approximately 1.1 km to approximately 250 m.

6.3 Greedy Algorithm Counterexample

The original counterexample used to demonstrate the limitation of the greedy resource assignment algorithm did not actually produce a different result under the implementation's boundary conditions.

Rather than retaining an incorrect example, the case was replaced with a six-job counterexample that genuinely demonstrates the limitation of the greedy approach.

6.4 Demonstration Data

The search demonstration initially used data that was already sorted because the records were loaded in primary-key order.

A deterministic shuffle was introduced so that the demonstration correctly represents an unsorted input before searching.

6.5 Trace Generation

Some trace-generation logic previously depended on hardcoded graph node IDs. Changes to the graph dataset could therefore cause the documentation-generation process to become invalid without producing an obvious error.

The trace-generation process was modified to discover the required nodes dynamically from the current dataset.

7. Challenges Encountered

7.1 Development Environment Constraints

During development, limited disk space on the development machine caused package installation and JVM allocation failures.
This affected the execution of some development and experimental tasks. The issue was mitigated by monitoring available disk space and reducing unnecessary dependencies, including the use of dependency-free SVG generation for the efficiency graphs.

7.2 Maintaining Consistency Between Data and Documentation

Changes to the dataset occasionally caused manually prepared demonstrations and traces to become outdated.

This highlighted the importance of generating evidence from the current dataset rather than relying on manually calculated examples.

As a result, several documentation-generation processes were automated.

8. Documentation Completed

The following project documentation has been prepared:

- Data dictionary
- Dataset evidence note
- Pseudocode documentation
- Trace tables
- Proof sketches
- Counterexamples
- Red-Black Tree walkthrough
- B-Tree traces
- Efficiency experiment results
- Efficiency graphs
- Final technical report draft
- Demonstration video script

The technical report has been structured according to the requirements of the project brief.

9. Outstanding Work

Although the major technical components of the project have been completed, several submission-related activities remain.

Task	Status
Technical implementation	**Complete**
Dataset and database	**Complete**
Data structures	**Complete**
Algorithms	**Complete**
Automated testing	**Complete**
Correctness evidence	**Complete**
Efficiency experiments	**Complete**
Technical report draft	**Complete**
Console demonstration screenshots	**Complete**
Machine specifications for experiments	**Complete**
Team member names and index numbers	**Complete**
Individual contribution statements	**Complete**
Cover-sheet information and signatures	**Complete**
Repository/ZIP information	**Complete**
Demonstration video	**Complete**

10. Overall Project Status

The project is currently technically complete and in the final documentation and demonstration stage.

All major implementation components have been completed, including the dataset, SQLite persistence layer, custom data structures, searching and sorting algorithms, graph algorithms, greedy resource assignment, dynamic programming, automated testing, and efficiency experiments.

The implementation has been validated using a 77-test automated test suite, with 77 out of 77 tests passing. Additional correctness and efficiency evidence has also been produced through trace tables, proof sketches, counterexamples, experiments, and performance graphs.

The remaining work does not involve major algorithmic or architectural development. It primarily consists of completing administrative information, finalizing the technical report, preparing screenshots, recording the demonstration, and completing the final submission package.

Individual Contribution Details

1. Precious Bassey Inyang - 22059210

Contribution: Implemented the graph structure using adjacency representations. Contributed BFS, DFS, Dijkstra, Prim and Kruskal operations for traversal, shortest paths and minimum-spanning-tree analysis.

2. Priscilla Kwakye – 22150093

Contribution: Implemented a hash table using separate chaining. Added insertion, retrieval, deletion, collision handling and load-factor functionality.

3. Edmund Mulcahy Amonoo - 22057575

Contribution: Implemented the self-balancing Red-Black Tree, including rotations, color adjustments, insertion and searching. The structure supports balanced-tree performance analysis.

4. Asante Kojo Boakye - 22065837

Contribution: Implemented the Binary Search Tree with insertion, searching, in-order traversal, size and height tracking. The BST also supports comparison with the RedBlack Tree.

5. Kwabena Ntiamoah Boateng - 22046431

Contribution: Implemented the array-based Min Heap Priority Queue with insertion, extract-min, heapify operations and dynamic capacity expansion for priority-based dispatch.

6. Owusu Sarfo Kwadwo - 22078485

Contribution: Implemented the multi-way B-Tree with searching, insertion, child splitting and root splitting. Split tracing was included for demonstrating tree balancing.

7. Daniel Okuluri Tetteh - 22297380

Contribution: Implemented Insertion Sort and Selection Sort and included them in the project's sorting and performance comparisons.

8. Daniel Kubi Adjetey Junior – 22059983
Contribution: Implemented Merge Sort and Quick Sort, including recursive partitioning/merging logic, and contributed to the sorting performance comparison.

9. Joel Asante - 22407542
Contribution: Implemented a resizable array-based double-ended queue supporting insertion and removal from both ends. It supports the project's critical-request prioritisation concept.

10. Daniel Ofori Boateng - 22408351
Contribution: Implemented the fixed-capacity Circular Queue using circular indexing, providing FIFO request processing and efficient wrap-around behaviour.

11. Prince Oppong - 22375768
Contribution: Implemented Binary Search for sorted arrays and demonstrated its logarithmic search behaviour, including the importance of the sorted-input requirement.

12. Christian Kwabena Asante - 22387357 z
Contribution: Implemented the arraybased Stack with push, pop, peek and dynamic expansion. Its LIFO behaviour supports the project's dispatch-reassignment/undo concept.

13. Evans Osei - 22411279
Contribution: Implemented the custom dynamic array list with insertion, retrieval, update, removal and automatic resizing, including resize tracing.

14. Gillis Abraham Nii Akwei McLutason - 22387934
Contribution: Implemented Linear Search for sequential searching without requiring sorted data and contributed to the comparison between linear and binary search.

15. Bryan Acquah Annor - 22368027
Contribution: Implemented the Doubly Linked List using bidirectional node links, supporting insertion, deletion, traversal and iterator operations.

Current status: ON TRACK FOR FINAL SUBMISSION